

50 Essential Git Interview Questions & Answers for Developers & DevOps Engineers

Introduction

Git is one of the most important tools used by Developers, DevOps Engineers, SREs, Cloud Engineers, and Software Teams worldwide. Understanding Git concepts is crucial for managing source code, collaborating with teams, and maintaining production-ready applications.

This document covers 50 frequently asked Git interview questions with concise answers to help you prepare for interviews and strengthen your practical Git knowledge.

1. What is Git and why is it used?

Git is a distributed version control system used to track changes in source code, collaborate with teams, and maintain project history.

2. Explain the difference between Git and GitHub.

- Git: Version control tool.
- GitHub: Cloud platform that hosts Git repositories and provides collaboration features.

3. How do you install Git on your machine?

Linux:

```
sudo apt install git
```

Windows: Download and install Git from the official website.

4. How do you configure your username and email in Git?

```
git config --global user.name "Your Name"  
git config --global user.email "your@email.com"
```

5. What is a repository in Git?

A repository (repo) is a storage location that contains project files, Git history, branches, and configurations.

6. How do you create a new Git repository?

```
git init
```

7. How do you clone a repository from GitHub?

```
git clone <repository-url>
```

8. What is the purpose of the .gitignore file?

It specifies files and folders Git should ignore and not track.

9. How do you check the status of your working directory in Git?

```
git status
```

10. How do you add files to the staging area in Git?

```
git add file.txt  
git add .
```

11. Explain the concept of commits in Git.

A commit is a snapshot of changes recorded in the repository history.

12. How do you create a new commit in Git?

```
git commit -m "Commit message"
```

13. What is the purpose of the git log command?

Displays commit history.

14. How do you view the history of commits in a repository?

```
git log
```

15. How do you view the changes made in a commit?

```
git show <commit-id>
```

16. What is branching in Git and why is it useful?

Branching allows parallel development without affecting the main codebase.

17. How do you create a new branch in Git?

```
git branch feature-branch
```

18. How do you switch between branches in Git?

```
git switch feature-branch
```

19. What is the difference between git merge and git rebase?

- Merge preserves history with a merge commit.

- Rebase creates a cleaner linear history.

20. How do you resolve merge conflicts in Git?

Edit conflicting files, remove conflict markers, then:

```
git add .  
git commit
```

21. What is the purpose of the git stash command?

Temporarily saves uncommitted changes.

22. How do you apply stashed changes in Git?

```
git stash apply
```

23. What is the purpose of the git tag command?

Creates version markers such as v1.0.0.

24. How do you create and push tags to a remote repository?

```
git tag v1.0.0  
git push origin v1.0.0
```

25. Explain the concept of remote repositories in Git.

Remote repositories are hosted versions of your Git project used for collaboration.

26. How do you add a remote repository in Git?

```
git remote add origin <repo-url>
```

27. How do you push changes to a remote repository?

```
git push origin main
```

28. How do you pull changes from a remote repository?

```
git pull origin main
```

29. What is the purpose of the git fetch command?

Downloads changes without merging them.

30. How do you delete a branch in Git?

```
git branch -d branch-name
```

31. Create a new Git repository and configure your username and email.

```
git init
git config user.name "John"
git config user.email "john@example.com"
```

32. Create a file, add some content to it, and commit the changes.

```
echo "Hello Git" > file.txt
git add .
git commit -m "Initial commit"
```

33. Create a .gitignore file and add rules.

```
echo "*.log" >> .gitignore
echo "node_modules/" >> .gitignore
```

34. Clone an existing repository from GitHub and make changes.

```
git clone <repo-url>
cd repo
```

Modify files and commit changes.

35. Create a new branch, make changes, and switch back.

```
git checkout -b feature
git checkout main
```

36. Merge changes from a feature branch into main.

```
git checkout main
git merge feature
```

37. Resolve a merge conflict between two branches.

Manually edit conflicts and commit resolved files.

38. Difference between working directory, staging area, and repository.

- Working Directory: Current files.
- Staging Area: Prepared changes.
- Repository: Committed history.

39. Difference between tracked and untracked files.

- Tracked: Git monitors changes.
- Untracked: Git doesn't monitor.

40. What does git init do internally?

Creates the hidden .git directory and initializes repository metadata.

41. What is the purpose of the .git directory?

Stores commits, objects, references, logs, and configuration.

42. Difference between git pull and git clone.

- Clone: Downloads repository first time.
- Pull: Updates existing repository.

43. What is git cherry-pick?

Applies a specific commit from one branch to another.

```
git cherry-pick <commit-id>
```

44. What is git revert and how is it different from git reset?

- Revert creates a new commit to undo changes.
- Reset moves branch pointers.

45. How do you squash commits?

```
git rebase -i HEAD~3
```

Choose squash option.

46. How do you undo the last commit without losing changes?

```
git reset --soft HEAD~1
```

47. How can you recover a deleted branch?

```
git reflog  
git checkout -b recovered-branch <commit-id>
```

48. How do you resolve conflicts during rebase?

Resolve conflicts manually and continue:

```
git rebase --continue
```

49. What is git blame used for?

Shows who modified each line of a file and when.

```
git blame file.txt
```

50. Explain different types of git reset.

Soft Reset

```
git reset --soft HEAD~1
```